

Teorie minimală — interface

Pornind de la codul tău: `Figura`, `Interval`, `Orar` · Java · Adrian

`abstract` răspundea la „ce ești”. `interface` răspunde la „ce poți face”. Toată diferența dintre ele pornește de aici.

1. De unde vine nevoia — tot din codul tău

Aceeași capacitate, în trei locuri care n-au nicio rudă comună

```
Figura    → public Figura duplicare()
Interval → public Interval duplicate()
Orar      → public Orar duplicate()
```

Toate trei fac același lucru: își produc o copie. Dar `Figura` și `Interval` n-au nicio superclasă comună, iar `Orar` nu e nici figură, nici interval — e lucrul care *ține* intervale.

Consecința: **nu poți scrie o singură metodă care duplică orice**. Ca s-o scrii, ți-ar trebui un tip comun pentru parametru — și nu ai unul.

Și nu-l poți obține cu `extends`

Prima idee ar fi o superclasă comună deasupra tuturor. Două motive pentru care nu merge:

- Un `Orar` n-are ce moșteni de la o `Figura`. Ai lega lucruri care n-au nimic împreună doar ca să ai un tip comun.
- Locul e adesea **deja ocupat**. `Eticheta` moștenește `Dreptunghi`; în Java o clasă are **un singur** părinte, deci orice altceva ar mai trebui să fie `Eticheta` nu mai încapă.

2. Ce este o interfață

O listă de metode pe care o clasă promite că le are

```
public interface Duplicabil {
    Duplicabil copie();
}

public class Cerc extends Figura implements Duplicabil { ... }
public class Orar implements Duplicabil { ... }
```

Atât. În interfață scrii **numele metodei, ce primește și ce întoarce** — fără corp. Clasa care scrie `implements` se obligă să le scrie pe toate, altfel nu compilează.

Ce **nu** are o interfață: câmpuri de instanță (nu țin starea unui obiect) și constructor. Nici ea nu se instanțiază — la fel ca o clasă abstractă.

Singura regulă de sintaxă pe care merită s-o ții minte

	extends	implements
câte poți avea	exact una	oricâte
ce primești	cod scris + câmpuri	doar obligația de a scrie
răspunde la	„ce sunt”	„ce pot face”

`Cerc` poate fi în același timp `Figura` (prin moștenire) și „duplicabil” și „cu arie” (prin două interfețe). Doar prima e limitată la una singură.

3. Ce câștigi, concret

Un tip comun acolo unde moștenirea nu poate ajunge

```
List<Duplicabil> lucruri = new ArrayList<>();
lucruri.add(new Cerc(...)); // din ierarhia de figuri
lucruri.add(new OraCurs(...)); // din ierarhia de orar
lucruri.add(new Orar()); // din nicio ierarhie

for (Duplicabil d : lucruri) {
    copii.add(d.copie());
}
```

Bucla nu întreabă niciodată ce are în față. Adugi mâine un tip complet nou, dintr-un pachet la care nici nu te gândeai — dacă scrie `implements Duplicabil`, intră în aceeași buclă fără nicio modificare.

Verificarea care nu se strică: „ce poți face”, nu „ce ești”

instanceof Cerc	instanceof AreArie
enumeri clasele, una câte una	întrebi o singură dată, despre capacitate
o clasă nouă cu arie e ignorată în tăcere	o clasă nouă cu arie e prinsă automat

De asta `instanceof` cu o interfață e acceptabil, iar `instanceof` cu un nume de clasă e aproape întotdeauna semnul că ți-a scăpat un polimorfism.

4. Interfață sau clasă abstractă?

	clasă abstractă	interfață
când tipurile...	sunt din aceeași familie	n-au nimic comun în afară de o capacitate
stare comună (câmpuri)	da	nu
cod scris o dată, folosit de toți	da	nu (la ce știm acum)
o clasă poate avea	una	oricâte

Nu sunt alternative care se exclud. `Cerc extends Figura implements Duplicabil`, `AreArie` e o linie perfect normală: moștenirea îi dă familia și punctele, interfețele îi dau capacitățile.

Cum decizi, în practică: dacă tipurile au **stare comună** și un „ce sunt” comun — clasă abstractă. Dacă au doar o **capacitate** comună, iar clasele vin din familii diferite (sau locul de `extends` e deja ocupat) — interfață. Întrebarea de control: pot să duc capacitatea asta la o clasă care n-are nicio legătură cu ierarhia mea? Dacă da, e interfață.